



Implementación y Comparación de Algoritmos de Búsqueda

ICC-708

Alumnos:

Juan Inzunza.

Carrera: Ingeniería Informática

1. Introducción

El presente informe describe el diseño e implementación de los algoritmos BFS, DFS, UCS y A* aplicados sobre dos tipos de escenarios: grafos con pesos y laberintos representados como matrices. Se detallan las decisiones de diseño adoptadas y se compara el comportamiento de cada algoritmo en términos de costo, nodos visitados y calidad del camino encontrado.

2. Decisiones de diseño

Todos los algoritmos reciben una función `get_vecinos(nodo) → [(vecino, costo)]` como parámetro. Este diseño desacopla la lógica de búsqueda del tipo de escenario: para el laberinto se devuelven las cuatro celdas libres adyacentes con costo 1; para el grafo, los nodos vecinos con su peso. Esto hace que los cuatro algoritmos operen sin conocer en qué escenario están corriendo.

BFS marca los nodos al encolarlos, evitando que el mismo nodo aparezca múltiples veces en la cola. DFS, en cambio, marca al desapilar, lo que refleja con mayor fidelidad el comportamiento de backtracking: un nodo puede estar en la pila más de una vez hasta que es procesado y descartado.

En relación a la heurística, en los laberintos se utiliza la distancia Manhattan: $h(n) = |fila_a - fila_b| + |col_a - col_b|$ como fue pedido.

Respecto a la visualización, el laberinto se construye con una matriz de colores usando `ListedColormap` y `BoundaryNorm` de `matplotlib`. Para el grafo se utiliza un dibujo manual con posiciones jerárquicas predefinidas, resaltando con color las aristas y nodos que forman el camino encontrado, nuevamente con `matplotlib`.

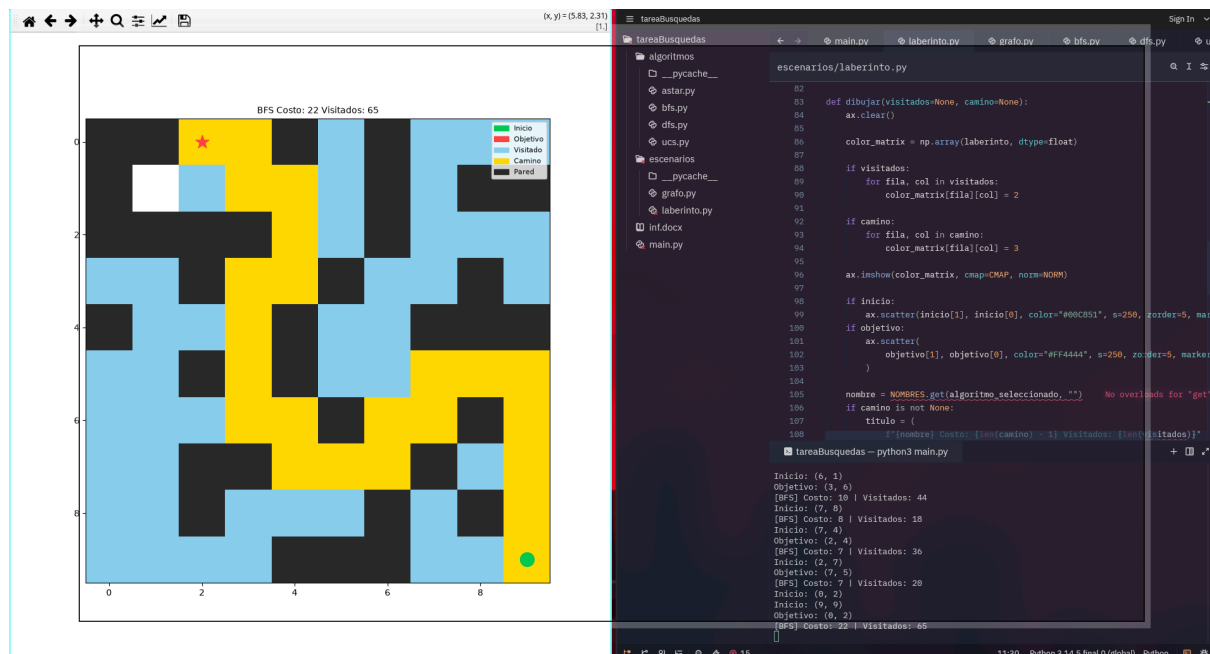
3. Comparación de algoritmos

La siguiente tabla resume las propiedades teóricas de cada algoritmo (b: factor de ramificación, d: profundidad de la solución, m: profundidad máxima, C*: costo óptimo, ϵ : costo mínimo de arista).

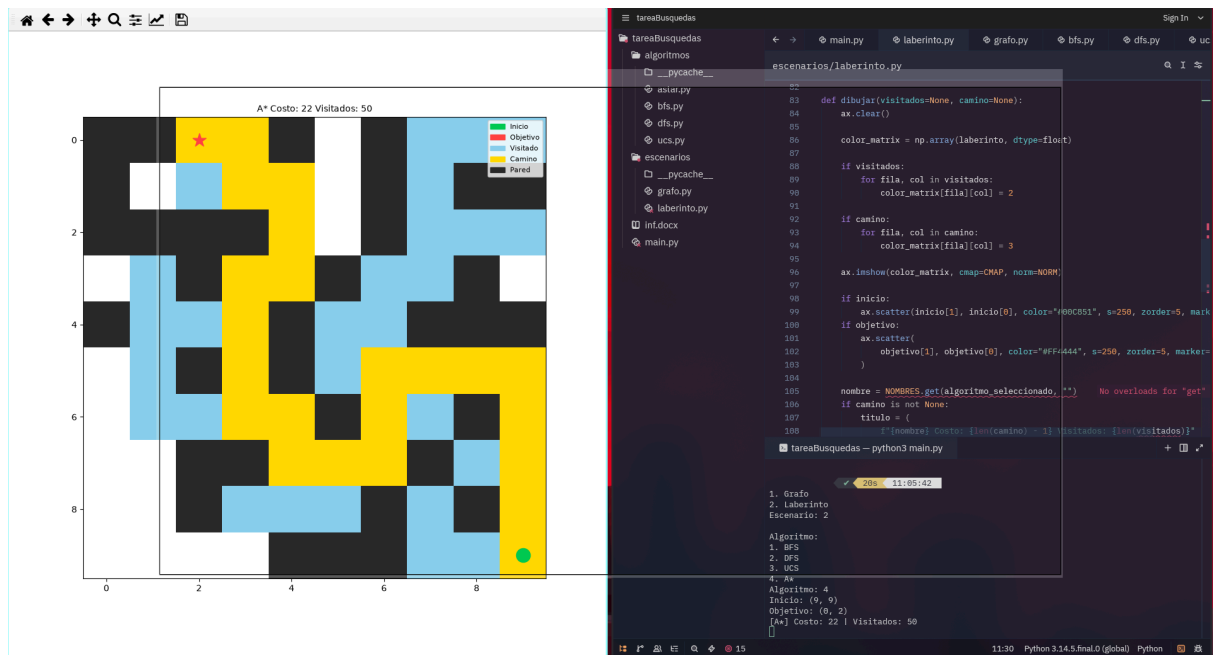
Algoritmo	Completo	Óptimo	Complejidad	Camino encontrado
BFS	Sí	Sí (pasos)	$O(b^d)$	Mínimo en número de pasos
DFS	Sí *	No	$O(b \cdot m)$	Puede ser subóptimo
UCS	Sí	Sí (costo)	$O(b^{C^*/\epsilon})$	Mínimo en costo acumulado
A*	Sí	Sí (h admis.)	Depende de $h(n)$	Mínimo en costo; menos nodos que UCS

Se ven implementados como:

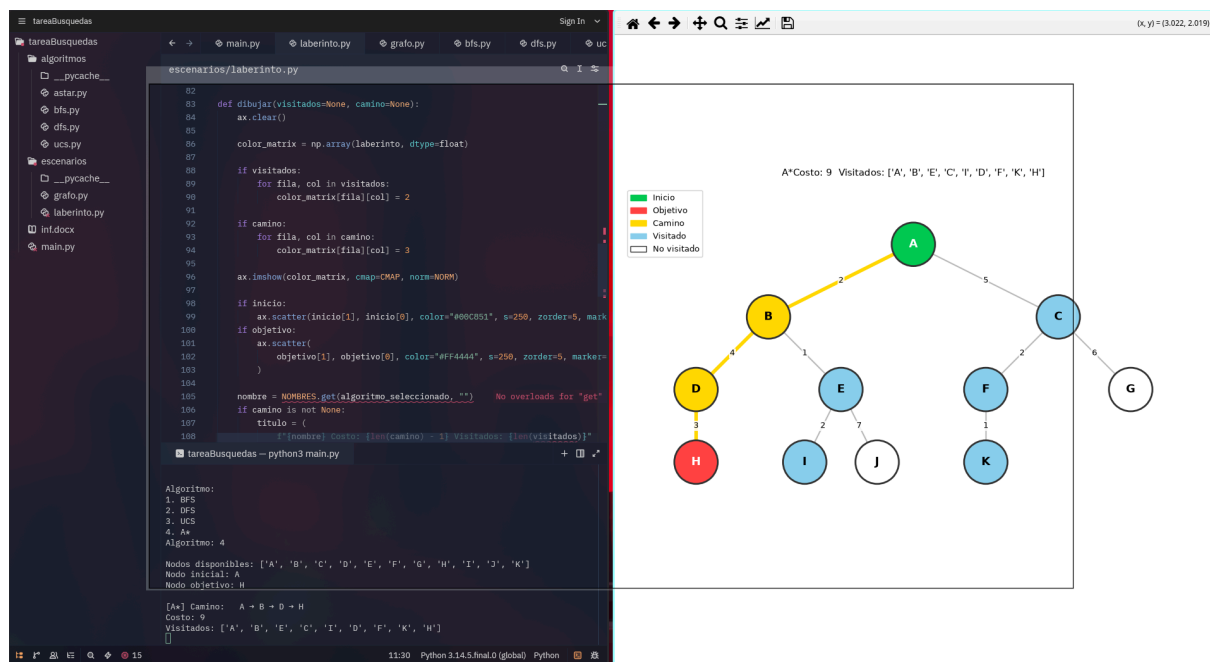
BFS - Laberinto



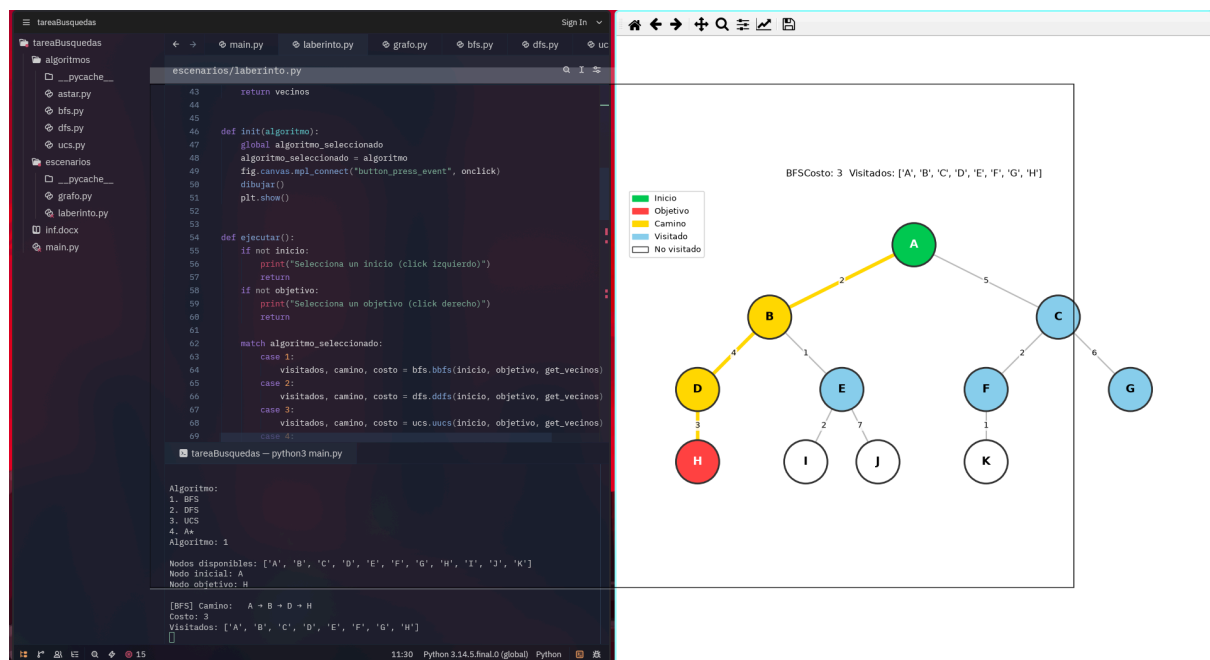
A* - Laberinto



A* - Grafo



BFS - Grafo



4. Conclusión

La implementación de los cuatro algoritmos sobre una interfaz común permitió verificar en la práctica las diferencias descritas por la teoría. BFS y UCS son las opciones más seguras cuando se requiere encontrar el objetivo sin importar el costo. DFS es preferible cuando la memoria es limitada y la calidad del camino no es crítica. A* ofrece el mejor balance entre eficiencia y optimalidad, siendo la elección ideal cuando se dispone de una buena heurística como la distancia Manhattan.